EE1D1: Digital Systems A

BSc. EE, year 1, 2026-2027, lecture 4

Introduction to SystemVerilog

Computer Engineering Lab

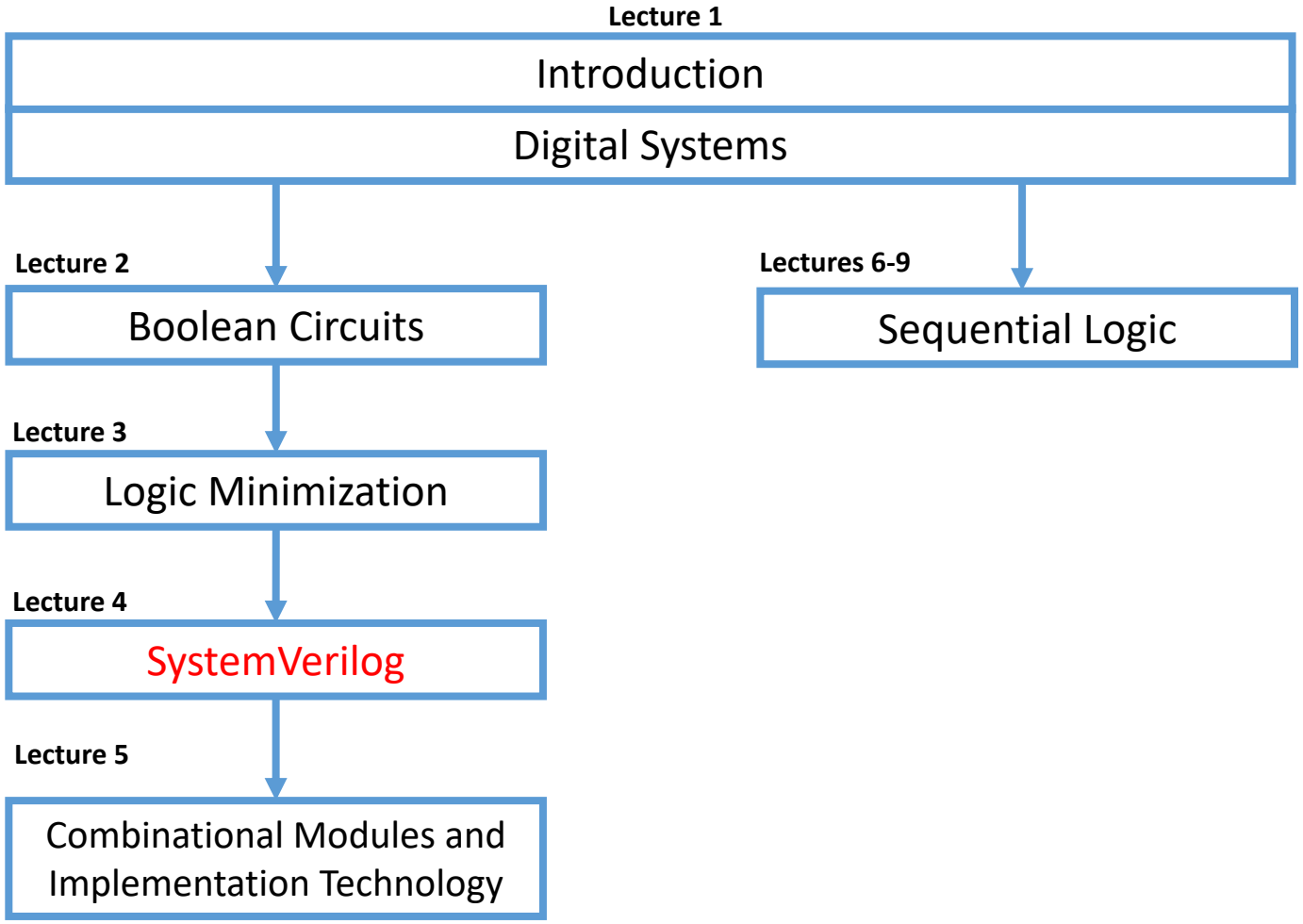
Faculty of Electrical Engineering, Mathematics & Computer Science

TU Delft

Delft University of Technology

Some figures and text
© 2021 Sarah Harris and David Harris

Recap


Lecture 1
Introduction
Digital Systems
Lecture 2
Boolean Circuits
Lecture 3
Logic Minimization
Lecture 4
SystemVerilog
Lecture 5
Combinational Modules and Implementation Technology
Lectures 6-9
Sequential Logic

Learning Objectives

As student you should be able to:

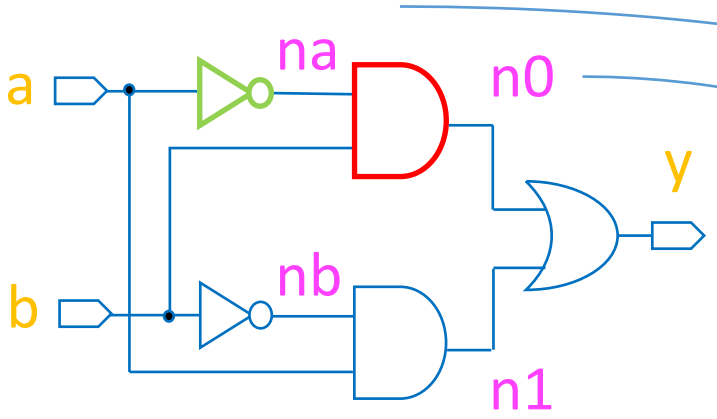
- Know what a hardware description language is used for.
- Describe a simple combinational circuit in SystemVerilog.
- Describe a structural (hierarchical) circuit in SystemVerilog.
- Understand how delay times are used during SystemVerilog simulation.
- Write a simple testbench for a combinational circuit in SystemVerilog.

Overview

- Intro Hardware Description Languages
- Modules
- Simulation and Synthesis
- Combinational Logic
- Numbers
- Delays
- Structural modeling
- Testbenches

Sections in book: 2.6.1, 4.1 - 4.3 (not tri-state buffer and value z), 4.9 (not example 4.39)

A Hardware Description Language



circuit diagram of
an exor circuit

```
module exor (input    logic a, b,
             output logic y);

  logic na, nb, n0, n1;

  assign na = ~a;
  assign n0 = na & b;
  assign nb = ~b;
  assign n1 = a & nb;
  assign y = n0 | n1;
endmodule
```

SystemVerilog description

Background

- Hardware description language (HDL):
  - Specifies logic function, at different abstraction levels.
  - Used to simulate the intended hardware.
  - Used to generate the final hardware (optimized netlist with all components).
- Most commercial designs built using HDLs.
- Two leading HDLs:
  - **SystemVerilog**
    - Developed in 1984 as Verilog by Gateway Design Automation, later extended to SystemVerilog.
  - **VHDL**
    - Developed in 1981 by the US Department of Defense
- We use SystemVerilog, not VHDL,
  - VHDL is more verbose and cumbersome

HDLs vs. Programming Languages

- Have syntactically similar constructs:
  - Data types, variables, assignments, if statements, loops, …
- But very different model: everything runs in parallel
  - Statement models hardware
  - Hardware is inherently parallel
- Hardware descriptions are composed of modules (mostly)
  - A hierarchy of modules connected to each other
  - Modules are active at the same time

IMPORTANT:

When using an HDL, think of the hardware the HDL should produce, then write the appropriate idiom that implies that hardware.

HDL to Gates

The two major purposes of HDLs are logic simulation and synthesis

- Simulation
  - Inputs applied to circuit
  - Simulator simulates the output signals
  - Outputs checked for correctness
  - Millions of euros saved by debugging in simulation instead of hardware
- Synthesis
  - Logic synthesizer transforms HDL code into a *netlist* describing the hardware (i.e., a list of gates and the wires connecting them)
  - Optimization is performed to reduce the amount of hardware.
  - The output netlist maybe a SystemVerilog file, or a schematic to visualize the circuit.

SystemVerilog in practice

- On Brightspace you can find information how to install the QuestaSim simulator and how to use it to simulate SystemVerilog descriptions
- On https://www.edaplayground.com/ you can do online simulations of SystemVerilog descriptions.
- You will practice SystemVerilog during the courselabs of DS-A and DS-B, and during the IP2 project.

SystemVerilog Modules

- A module is the basic building block in SystemVerilog
- Interfaces with outside using ***ports***

Example




```
module nand3(input  logic a, b, c,
             output logic y);

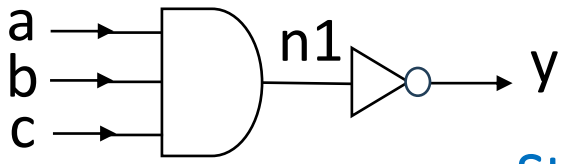
   // description of contents

endmodule
```

Ports/signals have a type, which is often **`logic`**. Values are **`0`**, **`1`**, **`x`** and **`z`**.

SystemVerilog Modules

Two types of contents descriptions within Modules:



- Behavioral: describe what the module does using programming language like constructs

```
module nand3(input  logic a, b, c,
             output logic y);
   logic n1;

   assign n1 = a & b & c;
   assign y = ~n1;
endmodule
```

n1 is an intermediate signal (node)

- Structural: describe how it is built from other modules

```
module nand3(input  logic a, b, c
             output logic y);
   logic n1;

   and3 i1 (a, b, c, n1);
   inv  i2 (n1, y);
endmodule
```

and3 and **inv** are modules that are described elsewhere.

They are called “instances within module nand3”.

i1 and **i2** are labels/instance names to distinguish different instances of the same module.

SystemVerilog Modules

In general, the order of statements is not important (they just describe contents).

Statements are *concurrent statements* that are *updated during simulation* whenever an input changes.

```
module nand3(input  logic a, b, c,
             output logic y);
   logic n1;

   assign n1 = a & b & c;
   assign y = ~n1;
endmodule
```



is equivalent to

```
module nand3(input  logic a, b, c,
             output logic y);
   logic n1;

   assign y = ~n1;
   assign n1 = a & b & c;
endmodule
```

```
module nand3(input  logic a, b, c
             output logic y);
   logic n1;

   and3 i1 (a, b, c, n1);
   inv  i2 (n1, y);
endmodule
```



is equivalent to

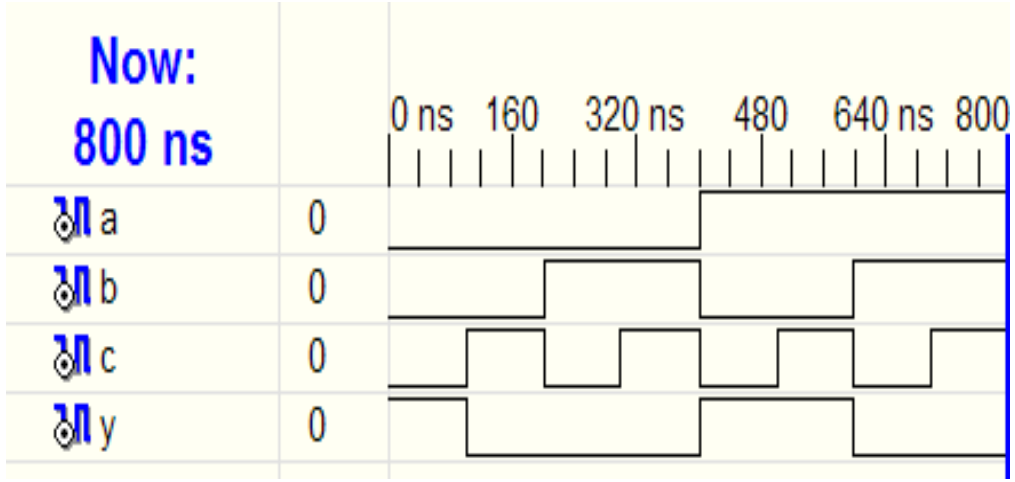
```
module nand3(input  logic a, b, c
             output logic y);
   logic n1;

   inv  i2 (n1, y);
   and3 i1 (a, b, c, n1);
endmodule
```

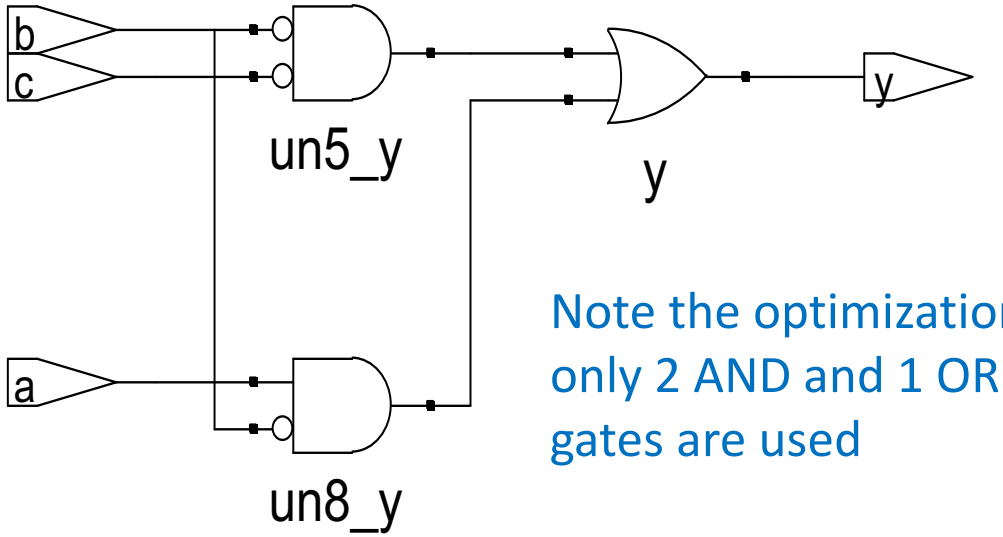
HDL Simulation & Synthesis

```
module example(input  logic a, b, c,
               output logic y);
   assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b &  c;
endmodule
```

Simulation Results



Synthesis Results



Note the optimization: only 2 AND and 1 OR gates are used

SystemVerilog Syntax

- **Case sensitive**
  - Example: `reset` and `Reset` are not the same signal.
  - Be consistent in your use of capitalization and underscores in signal and module names.
- **No** names that **start with numbers**
  - Example: `2mux` is an **invalid** name
- **Whitespace (spaces, tabs, newlines) are ignored**
  - Nevertheless, proper indenting and use of blank lines is helpful to make the code readable.
- **Statements are ended with a ;**
- **Comments:**
  - `// single line comment`
  - `/* multiline`
    `comment */`

Combinational Logic

Bitwise Operators

Bitwise operators are

Op	Meaning
~	NOT
&	AND
^	XOR
|	OR

E.g.

```
assign y = a & ~b | ~a & c;
```

means

```
y = (a AND (NOT b)) OR ((NOT a) AND c))
```

Signals

- Signals (ports and internal nodes) can be scalar (single-bit) or vector (multi-bit signals):

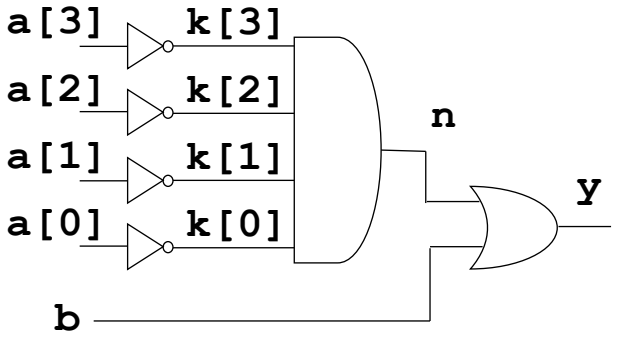
```
module func1(input logic [3:0] a,    // creates a[3], a[2], a[1] and a[0]
             input logic b,
             output logic y);

   logic [3:0] k;  // creates k[3], k[2], k[1] and k[0]
   logic n;

   assign k = ~a;  // implies k[3] = ~a[3], k[2] = ~a[2] etc.
   assign n = k[3] & k[2] & k[1] & k[0];
   assign y = b | n;

endmodule
```

Equivalent circuit

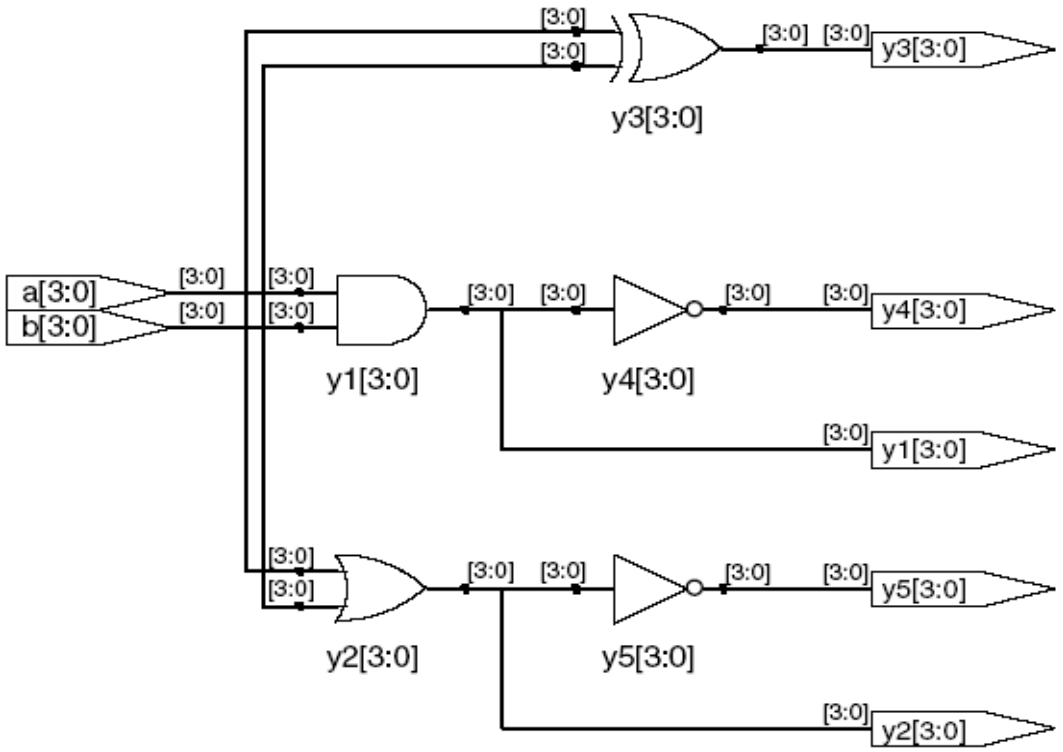


Bitwise Operators

- *Bitwise* operators act on single-bit signals or on multi-bit busses:

```
module gates(input  logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
   /* Five different two-input logic
      gates acting on 4 bit busses */
   assign y1 = a & b;    // AND
   assign y2 = a | b;    // OR
   assign y3 = a ^ b;    // XOR
   assign y4 = ~(a & b); // NAND
   assign y5 = ~(a | b); // NOR
endmodule
```

Synthesis:



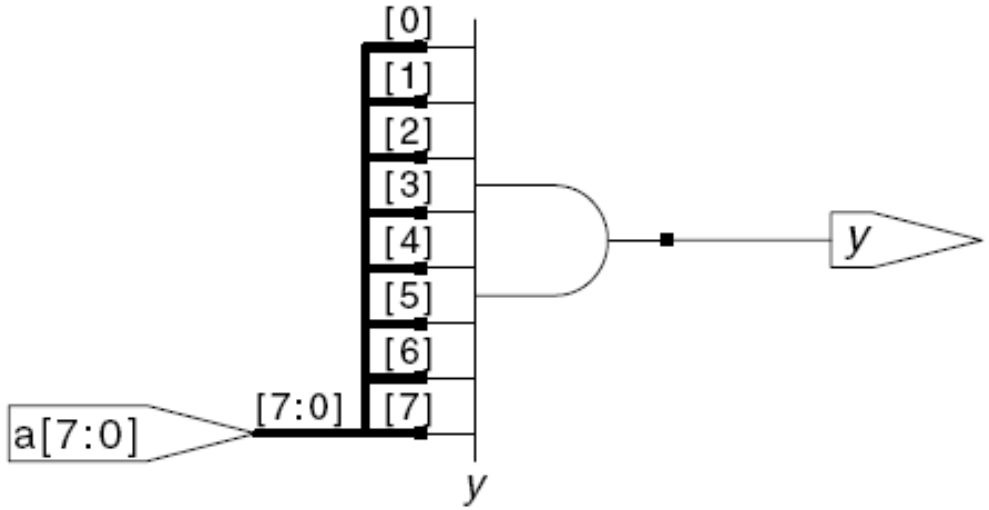
Reduction Operators

- *Reduction* operators imply a multiple-input gate acting on a single bus.

SystemVerilog:

```
module and8(input  logic [7:0] a,
            output logic       y);
   assign y = &a;
   // &a is much easier to write than
   // assign y = a[7] & a[6] & a[5] & a[4] &
   //            a[3] & a[2] & a[1] & a[0];
endmodule
```

Synthesis:



© 2021 Sarah Harris and David Harris

Conditional Assignment

- *Conditional assignments* select the output from alternatives based on an input called the *condition.*

```
module mux2(input  logic [3:0] d0, d1,
            input  logic       s,
            output logic [3:0] y);

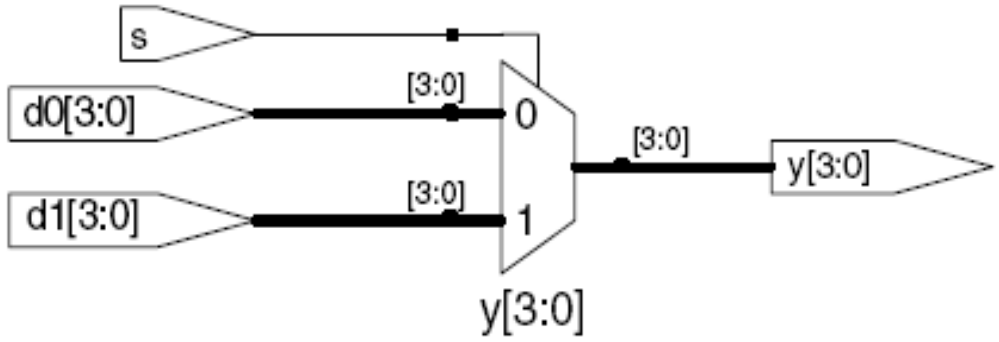
   assign y = s ? d1 : d0;

endmodule
```

If s is 1,then y = d1 else y = d0.

? : is also called a *ternary operator* because it operates on 3 inputs: **s, d1** and **d0.**

Synthesis:



This is a so-called multiplexer that will be further discussed in a next lecture.

Internal Variables

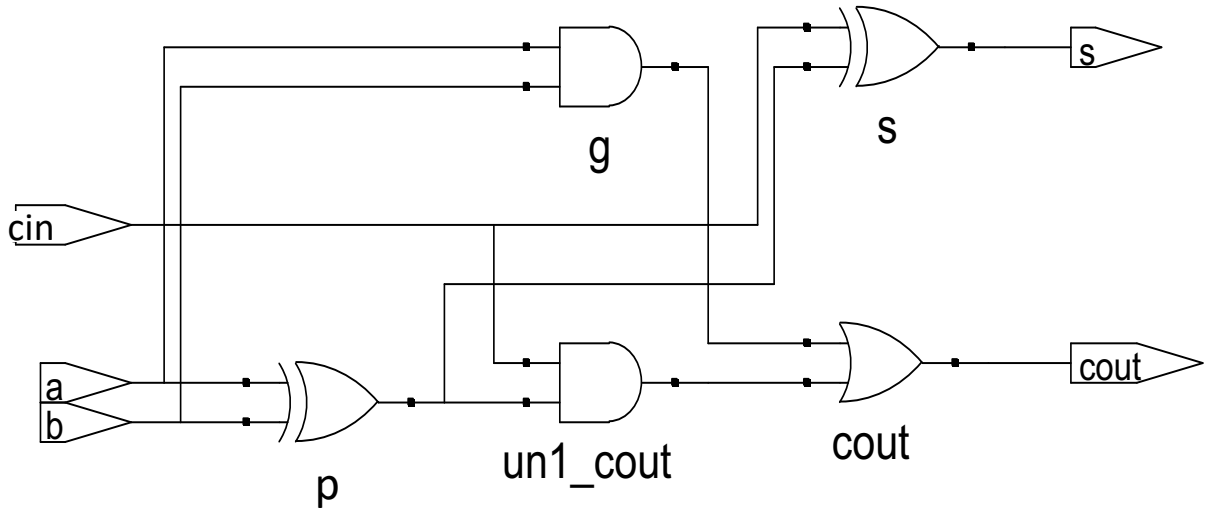
- Often it's convenient to break a complex function into intermediate steps and introduce internal variables, or nodes.

```
module fulladder(input  logic a, b, cin,
                 output logic s, cout);
  logic p, g;   // internal nodes

  assign p = a ^ b;
  assign g = a & b;

  assign s = p ^ cin;
  assign cout = g | (p & cin);
endmodule
```

Synthesis:



During synthesis, additional internal nodes (e.g. un1_cout) may be created

Operator precedence

Highest

Op	Meaning
~	NOT
*, /, %	mult, div, mod
+, -	add, sub
<<, >>	shift
<<<, >>>	arithmetic shift
<, <=, >, >=	comparison
==, !=	equal, not equal
&, ~&	AND, NAND
^, ~^	XOR, XNOR
|, ~|	OR, NOR
?:	ternary operator

Lowest

So **`~a | b & c`** is evaluated as **`(~a) | (b & c)`**

Numbers

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Used for logic vectors.

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00…0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hexadecimal	171	10101011
42	unsized	decimal (default)	42	00…0101010

Underscores (_) are used for formatting only to make it easier to read. SystemVerilog ignores them.

Numbers Examples

```
logic [7:0] k;
logic y;

// equivalent assignments
assign k = 8'b00101111;
assign k = 8'h2F;
assign k = 8'd47;
assign k = 47;

assign y = (k > 30)? 1 : 0;   // if k > 30 then y becomes 1 else 0
```

Bit Manipulations

Concatenation of a vector signal:

```
assign y = {a[3:1], 8'b1000_1010, a[0]};
```

If **y** is a 12-bit signal, the above statement produces:

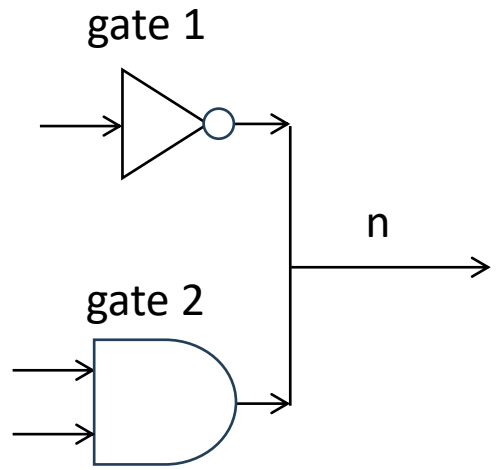
```
y = a[3] a[2] a[1] 1 0 0 0 1 0 1 0 a[0]
```

0, 1 and X

- Scalar (non vector) signals of type **`logic`** can have 4 different values:
  - **`0`**
  - **`1`**
  - **`x`** not initialized or contention (conflict)
  - **`z`** high impedance / floating value / not connected to voltage source (will be discussed with digital systems B).

- Besides value x being used as initial value at the start of the simulation, the example on the right shows how x is also used as conflict value.

If (by accident) 2 gate outputs are connected together.



Resulting value of node n as a function of the 2 gate outputs:

gate 2 \ gate 1	0	1
0	0	x
1	x	1

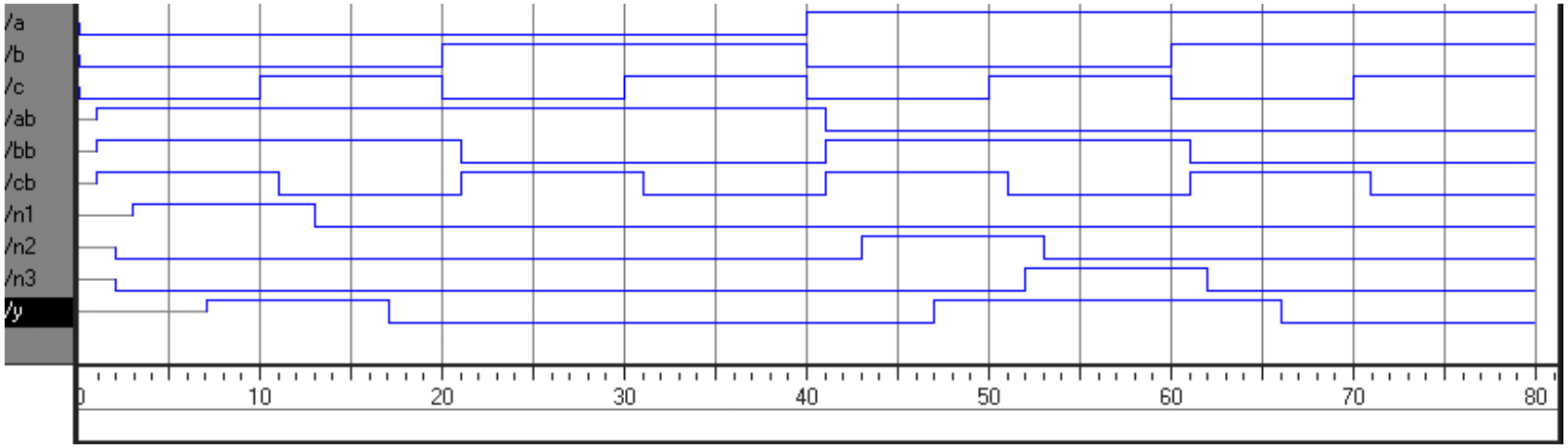
Delays

Delays

```
module example(input  logic a, b, c,
               output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

- The delay for each assignment is specified (e.g. #2): time between change of input and update of output.
- Delays are for simulation only! They do not determine the delay of the circuit after synthesis.

Simulation



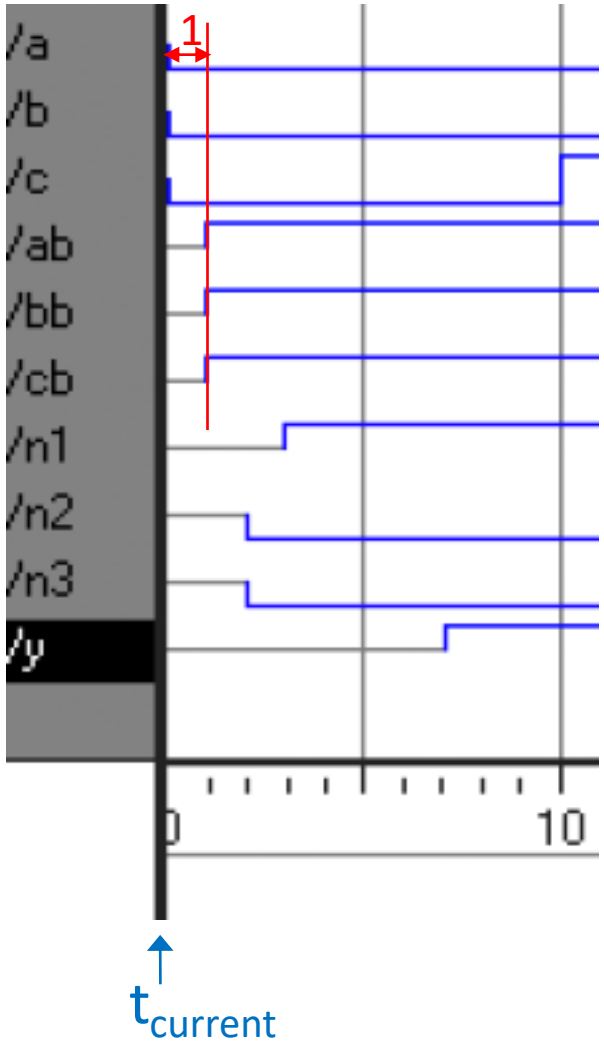
Delays

Assume simulation starts at $t_{current}$=0
with a, b, c going from x to 0.

At $t_{current}$=0, events on a, b, c trigger future updates at t=1

```
module example(input  logic a, b, c,
                      output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

Simulation

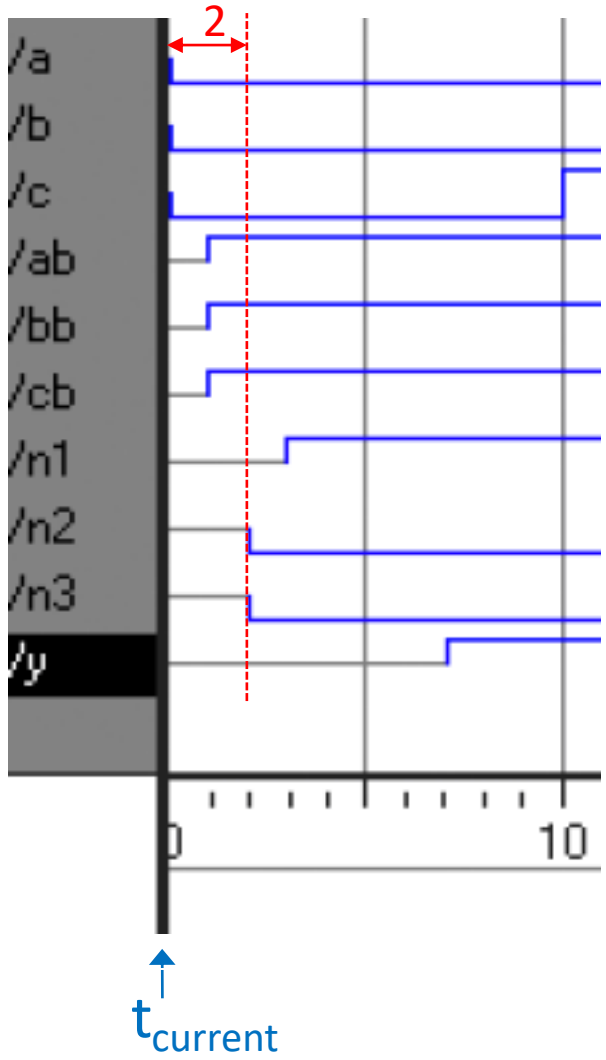


Delays

In addition, at $t_{current}$=0, events on a and c trigger future updates at t=2.

```
module example(input  logic a, b, c,
               output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

Simulation

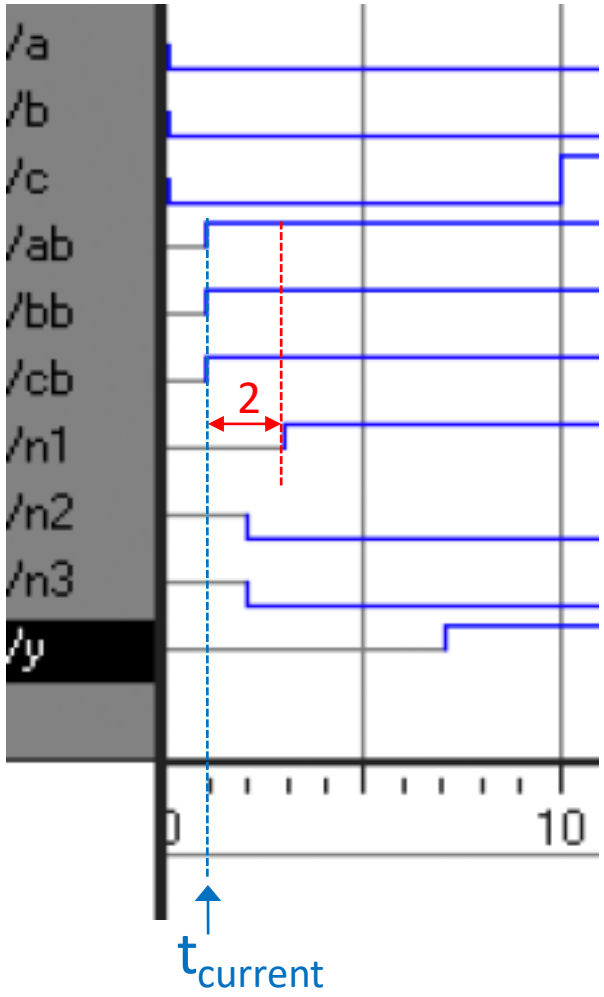


Delays

Next, simulation time $t_{current}$ is advanced to 1 and the events on ab, bb, cb at t=1 trigger future update at t=3.

```
module example(input  logic a, b, c,
               output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

Simulation

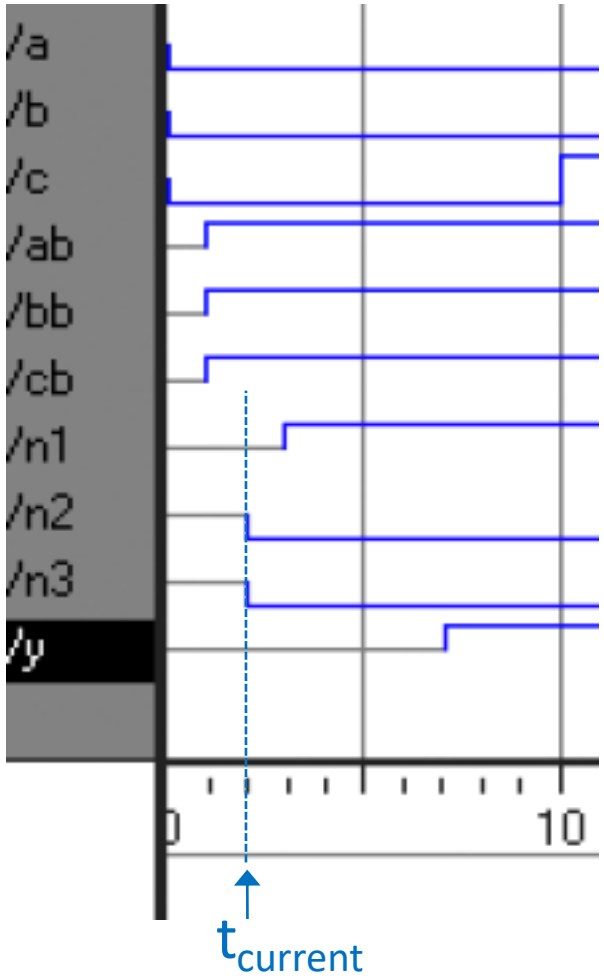


Delays

At $t_{current}$=2, the events on n2 and n3 do no trigger anything because y = (n1 | n2 | n3) = (x | 0 | 0) is still x.

```
module example(input  logic a, b, c,
               output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

Simulation

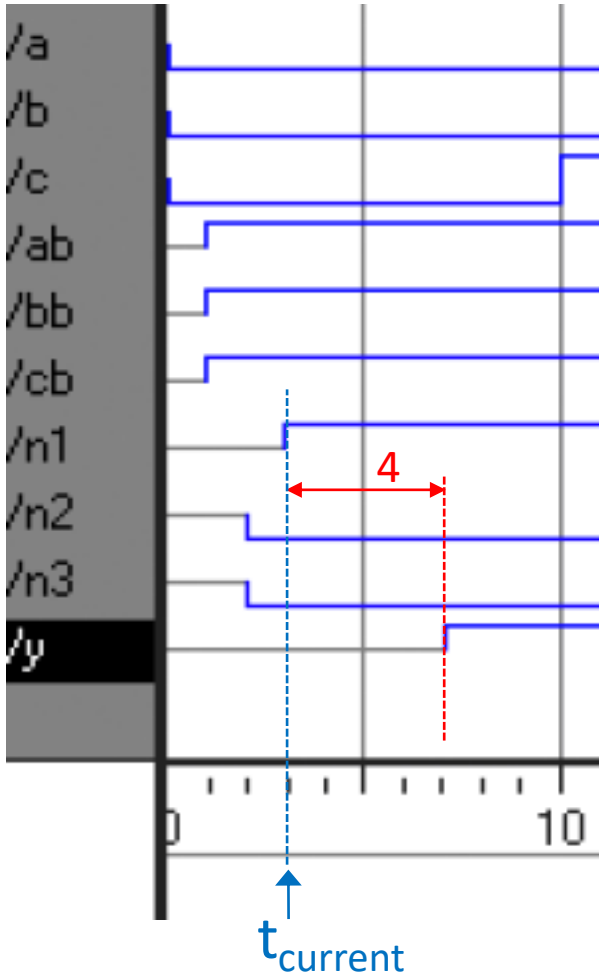


Delays

At $t_{current}$=3, the event on n1 triggers future update at t=7.

```
module example(input  logic a, b, c,
                     output logic y);
  logic ab, bb, cb, n1, n2, n3;
  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

Simulation



Structural Modeling

Structural SystemVerilog

- The previous sections discussed behavioral modeling: describing a module in terms of the relationships between inputs and outputs.
- This section examines structural modeling: describing a module in terms of how it is composed of simpler modules (hierarchy!).
- At the lowest level (leaf modules) you always need to use behavioral modelling.

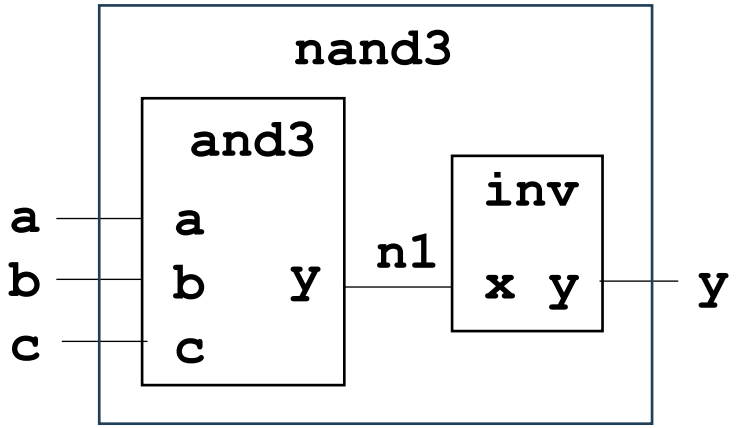
Example

```
module and3(input  logic a, b, c,
            output logic y);
  assign y = a & b & c;
endmodule

module inv(input  logic x,
           output logic y);
  assign y = ~x;
endmodule

module nand3(input  logic a, b, c,
             output logic y);
  logic n1;                          // internal signal

  and3 i1(a, b, c, n1);  // instance of and3
  inv  i2(n1, y);        // instance of inv
endmodule
```



Structural SystemVerilog

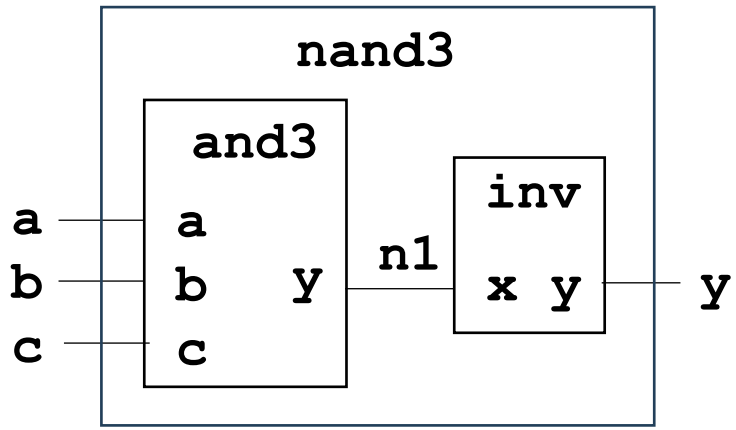
- Alternative specification of port connections.
- *Named port connections* instead of *ordered port connections*
- Order of port connections can now be different.

```
module and3(input  logic a, b, c,
            output logic y);
  assign y = a & b & c;
endmodule

module inv(input  logic x,
           output logic y);
  assign y = ~x;
endmodule

module nand3(input  logic a, b, c,
             output logic y);
  logic n1;

  and3 i1(.a(a), .b(b), .c(c), .y(n1));
  inv  i2(.x(n1), .y(y));
endmodule
```

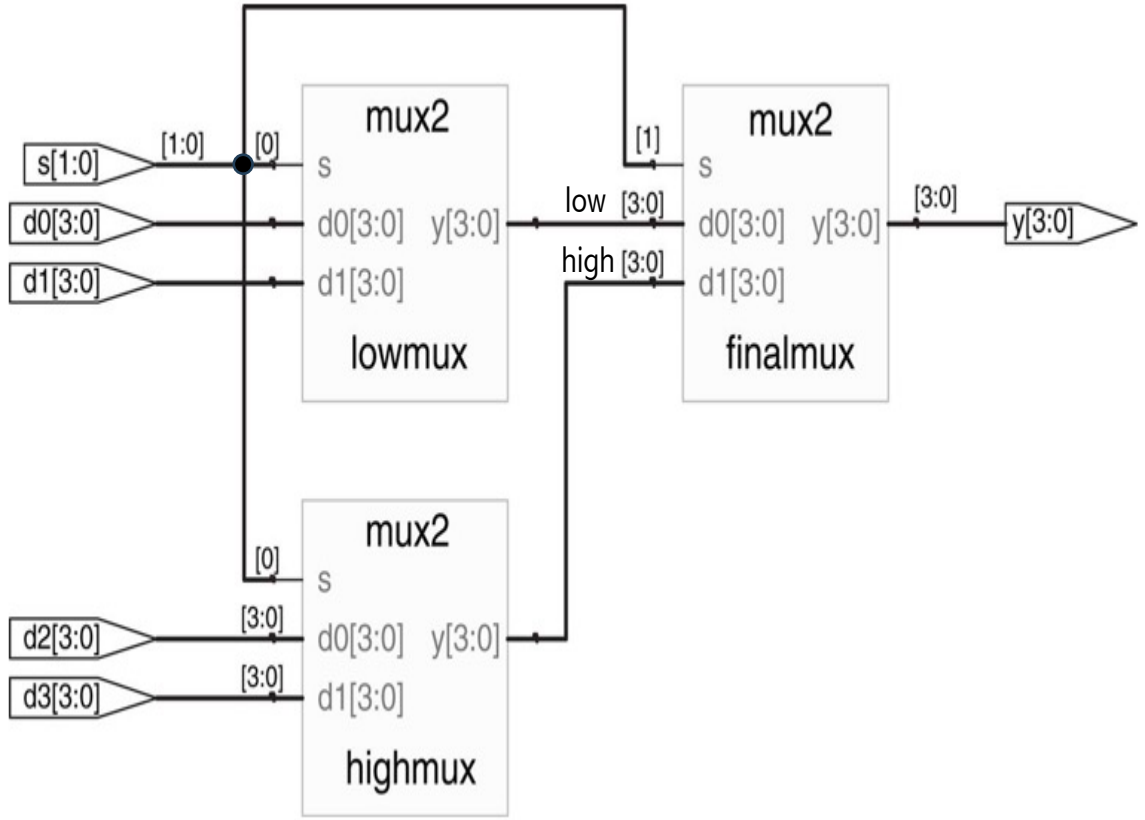


Structural Modeling: 4:1 Multiplexer

```
module mux4(input logic [3:0] d0, d1, d2, d3,
            input logic [1:0] s,
            output logic [3:0] y);

   logic [3:0] low, high;

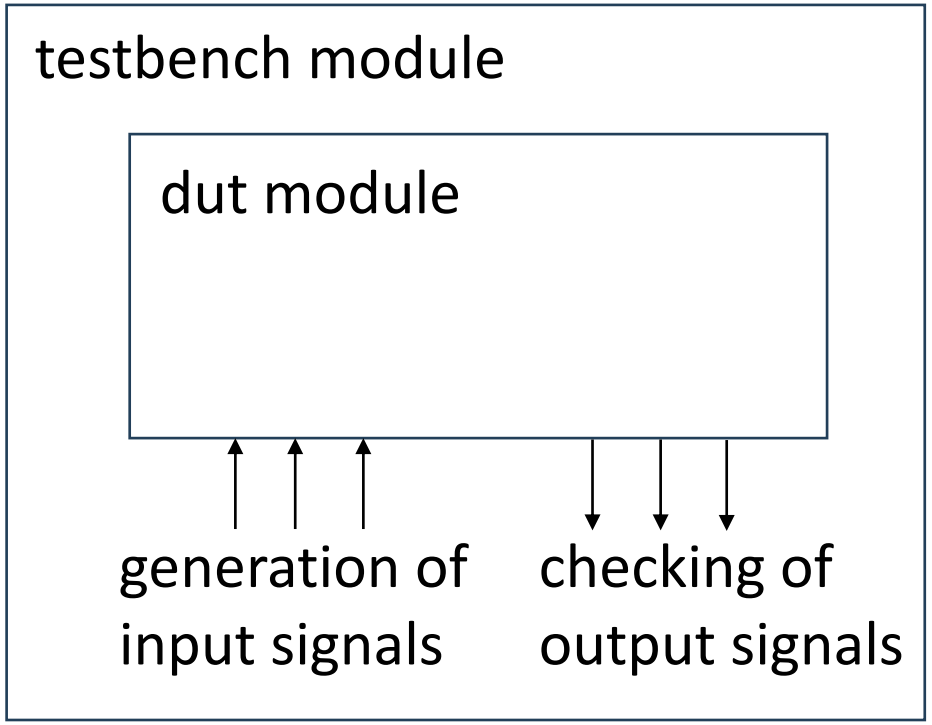
   mux2 lowmux(d0, d1, s[0], low);
   mux2 highmux(d2, d3, s[0], high);
   mux2 finalmux(low, high, s[1], y);
endmodule
```



Testbenches

Testbenches

- HDL module that **tests another module**: *device under test* (dut)
- **Not synthesizeable**



Device Under Test

Let's make a testbench to test a module **`sillyfunction`** that implements the logic function

$$y = b'c' + a\,b'$$

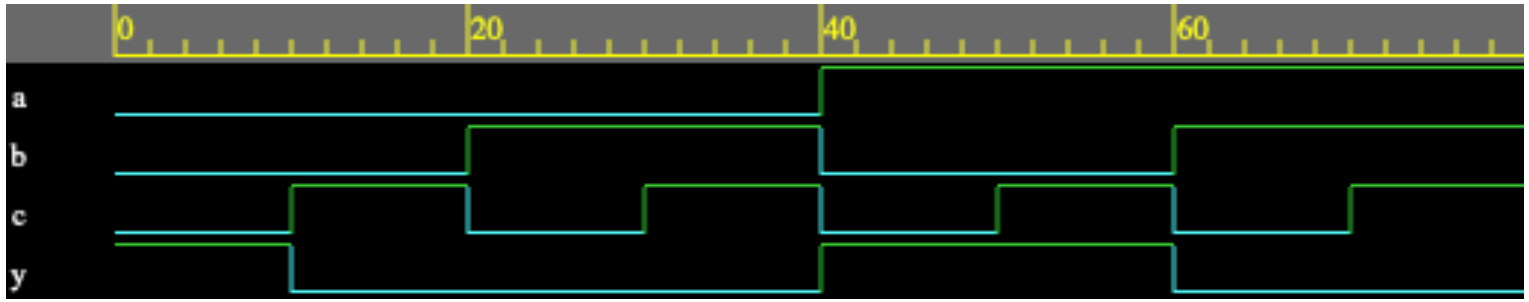
```
module sillyfunction(input  logic a, b, c,
                     output logic y);
  assign y = ~b & ~c | a & ~b;
endmodule
```

Testbench

```
module testbench1();
  logic a, b, c;
  logic y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply new set of inputs every 10 time-units
  initial begin
    a = 0; b = 0; c = 0; #10;
                  c = 1; #10;
           b = 1; c = 0; #10;
                  c = 1; #10;
    a = 1; b = 0; c = 0; #10;
                  c = 1; #10;
           b = 1; c = 0; #10;
                  c = 1; #10;
  end
endmodule
```

Simulation result

```
module testbench1();
  logic a, b, c;
  logic y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply new set of inputs every 10 time-units
  initial begin
    a = 0; b = 0; c = 0; #10;
                  c = 1; #10;
           b = 1; c = 0; #10;
                  c = 1; #10;
    a = 1; b = 0; c = 0; #10;
                  c = 1; #10;
           b = 1; c = 0; #10;
                  c = 1; #10;
  end
endmodule
```



Thank you